

Perceptron

Implementação e desempenho

Hebert A. Kubis





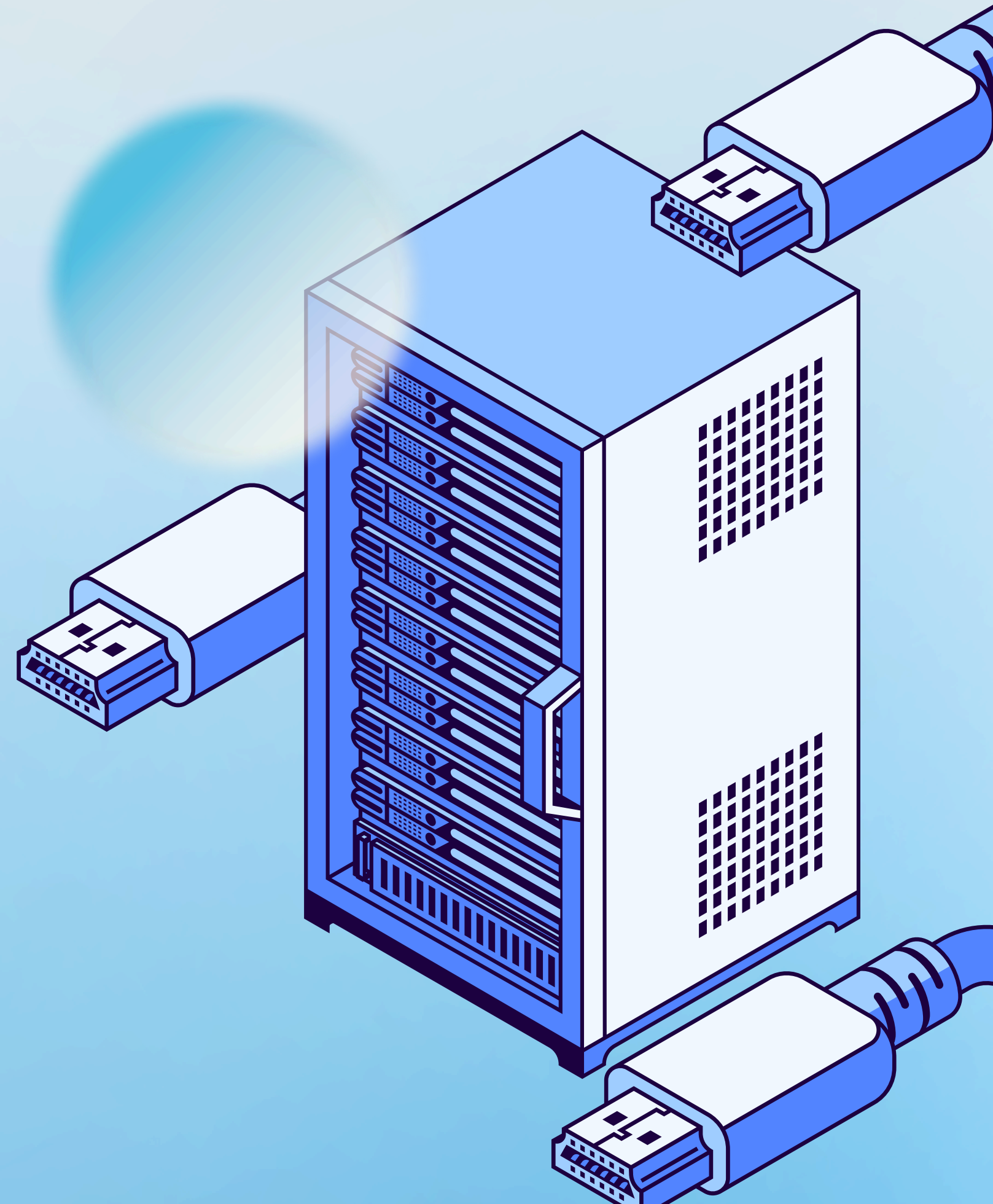
Implementação

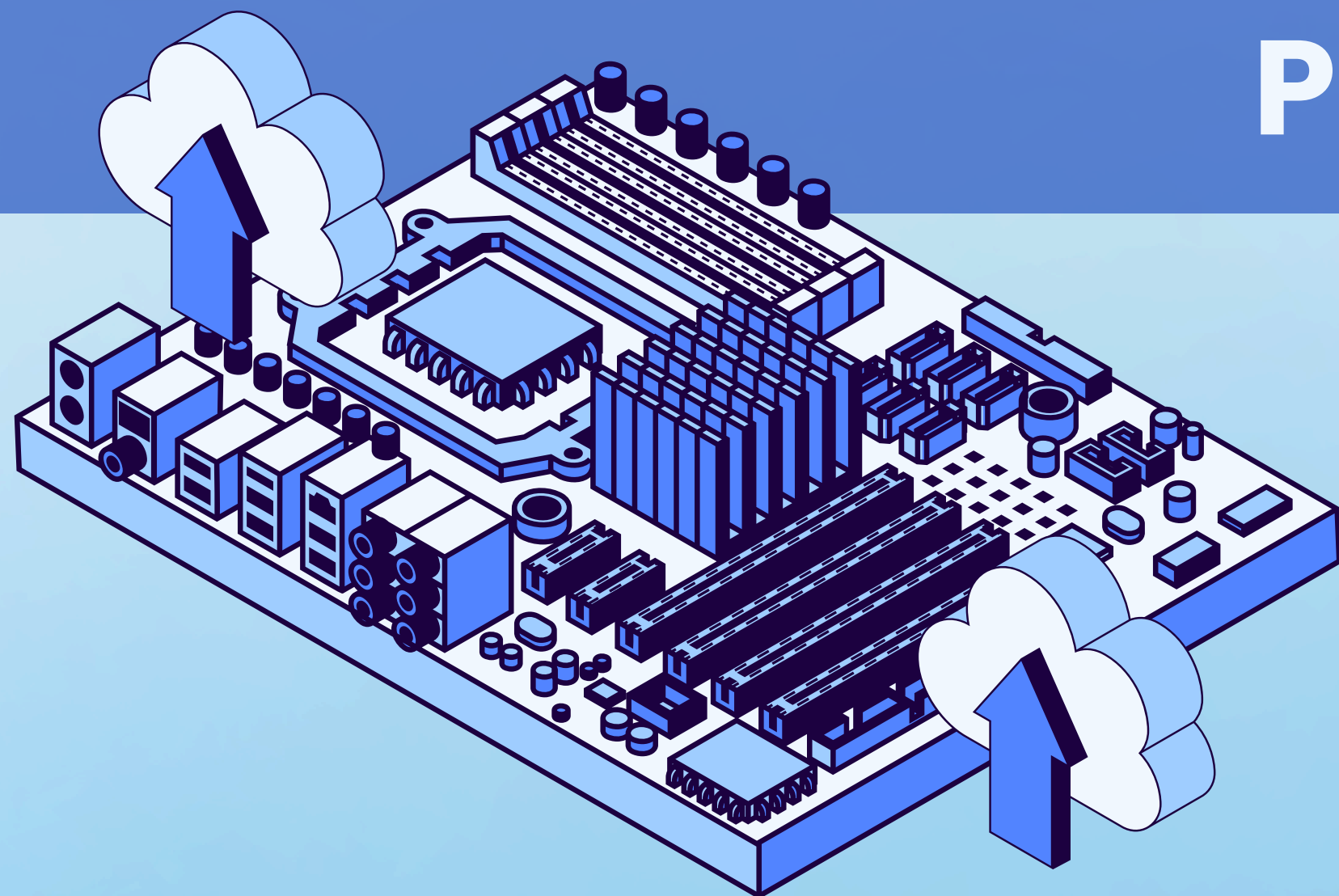
Feita em C, se baseando no paradigma de programação orientada a objetos. Foram utilizadas estruturas para representar tanto cada neurônio individual quanto o Perceptron como um todo e funções que modificam e utilizam as variáveis dessas estruturas.

Metodologia de validação

Para o treinamento e validação da rede foi utilizada a metodologia de validação Hold-out, separando $\frac{2}{3}$ dos dados para treinamento e $\frac{1}{3}$ para teste.

Os dados de treinamento foram embaralhados a cada época de treinamento para garantir que a rede não decorasse padrões.





Parâmetros da Rede

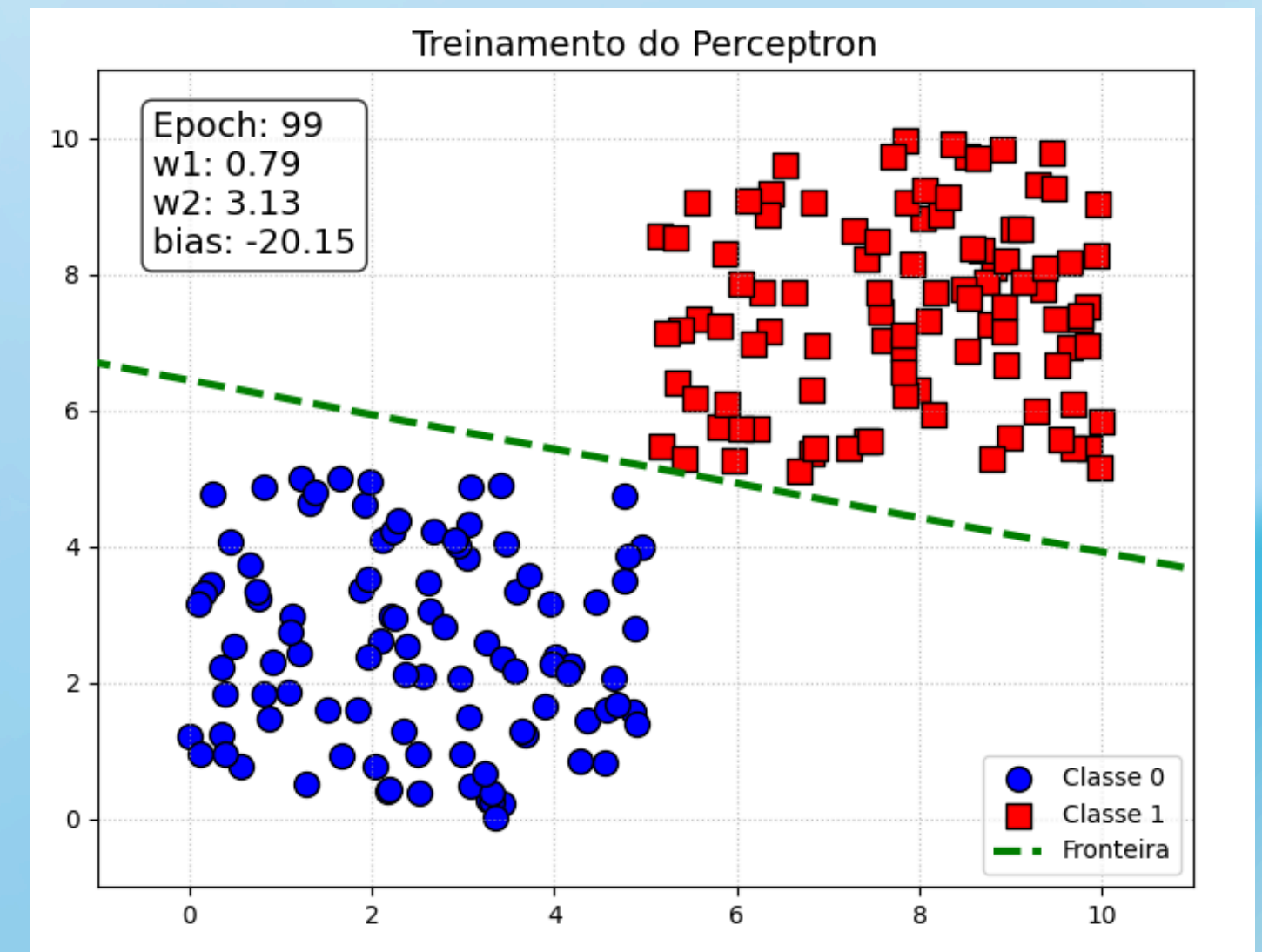
Para todos os testes feitos com a rede foram utilizados os mesmos parâmetros:

- **Taxa de aprendizado (η):** 0.65
- **Arquitetura:** 1 neurônio de saída
- **Função de ativação:** Função degrau unitário
- **Critério de parada:** Número máximo fixo de 100 epochs

*A rede foi configurada para ter um neurônio na saída pela classificação ser binária, mas a mesma foi construída de modo a ser possível configurar o número de neurônios na saída de acordo com a quantidade de classes.

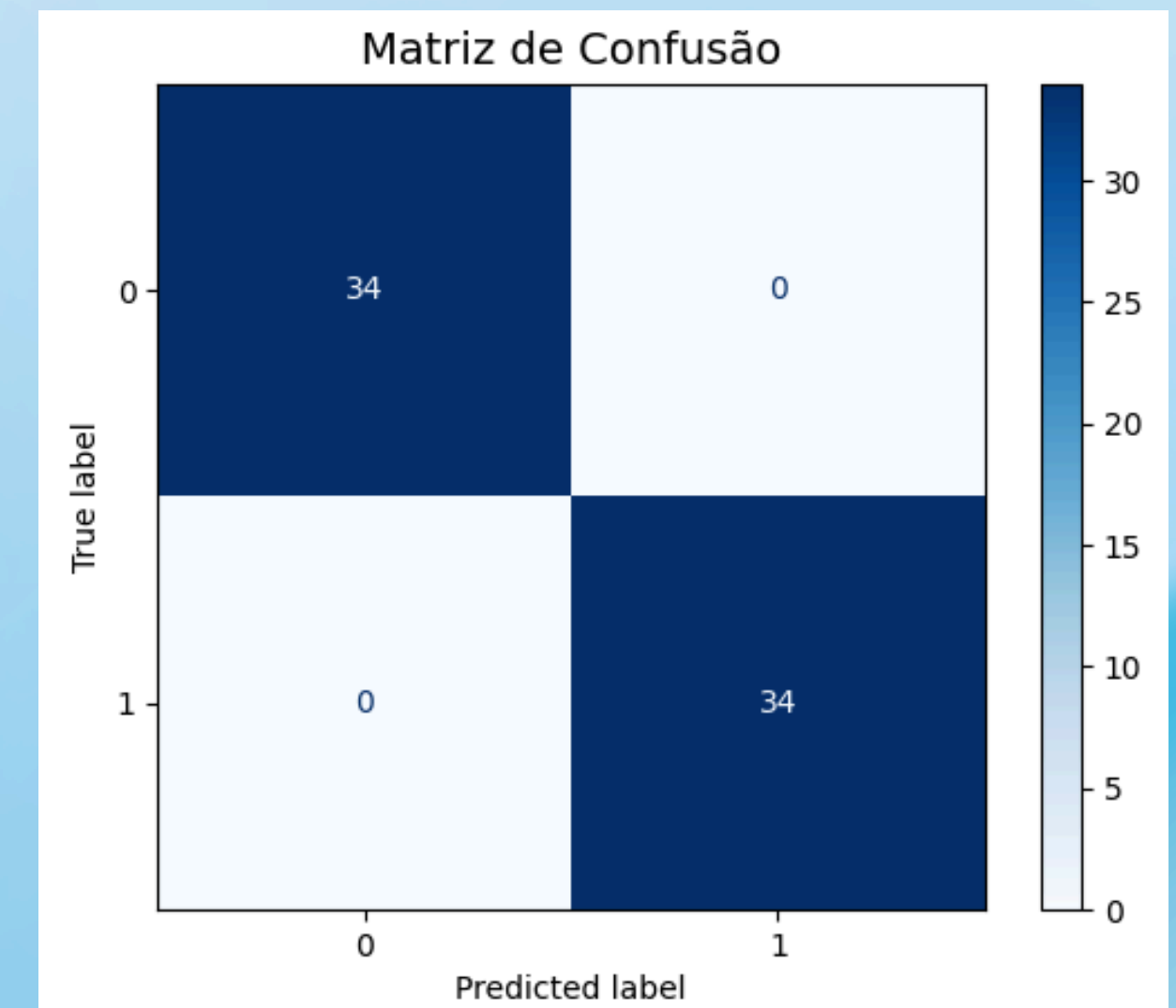
Base Linearmente Separável

- Criada sinteticamente por um programa em python que gerou pontos em dois quadrantes opostos no plano.
- Ajustou a fronteira de decisão já na segunda epoch



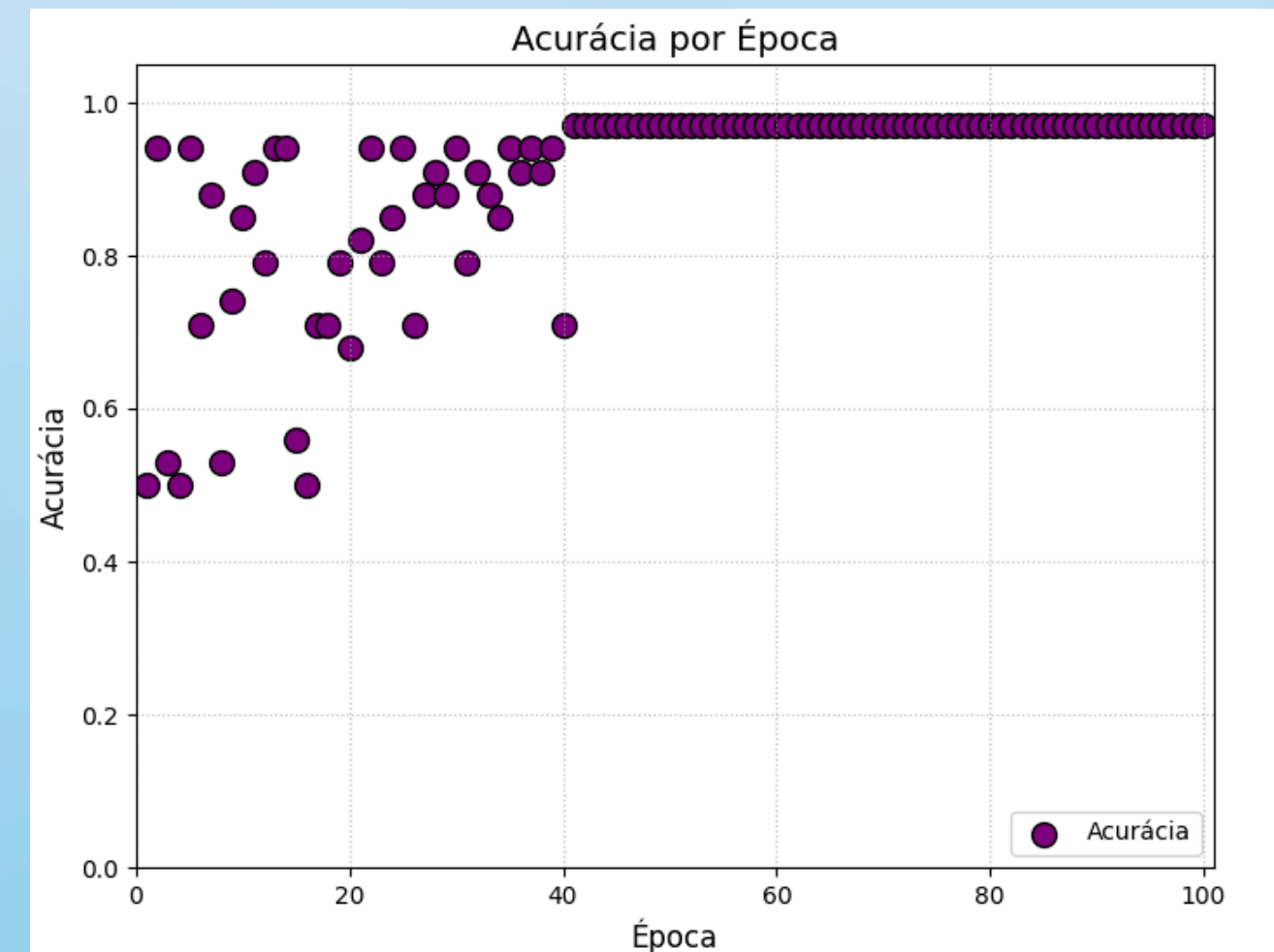
Desempenho na base

- Com ajuda da biblioteca Scikit-learn e Matplotlib é possível verificar a matriz de confusão gerada para os dados de teste.
- O gráfico de acurácia por epoch foi omitido por a rede ter convergido já na segunda epoch.

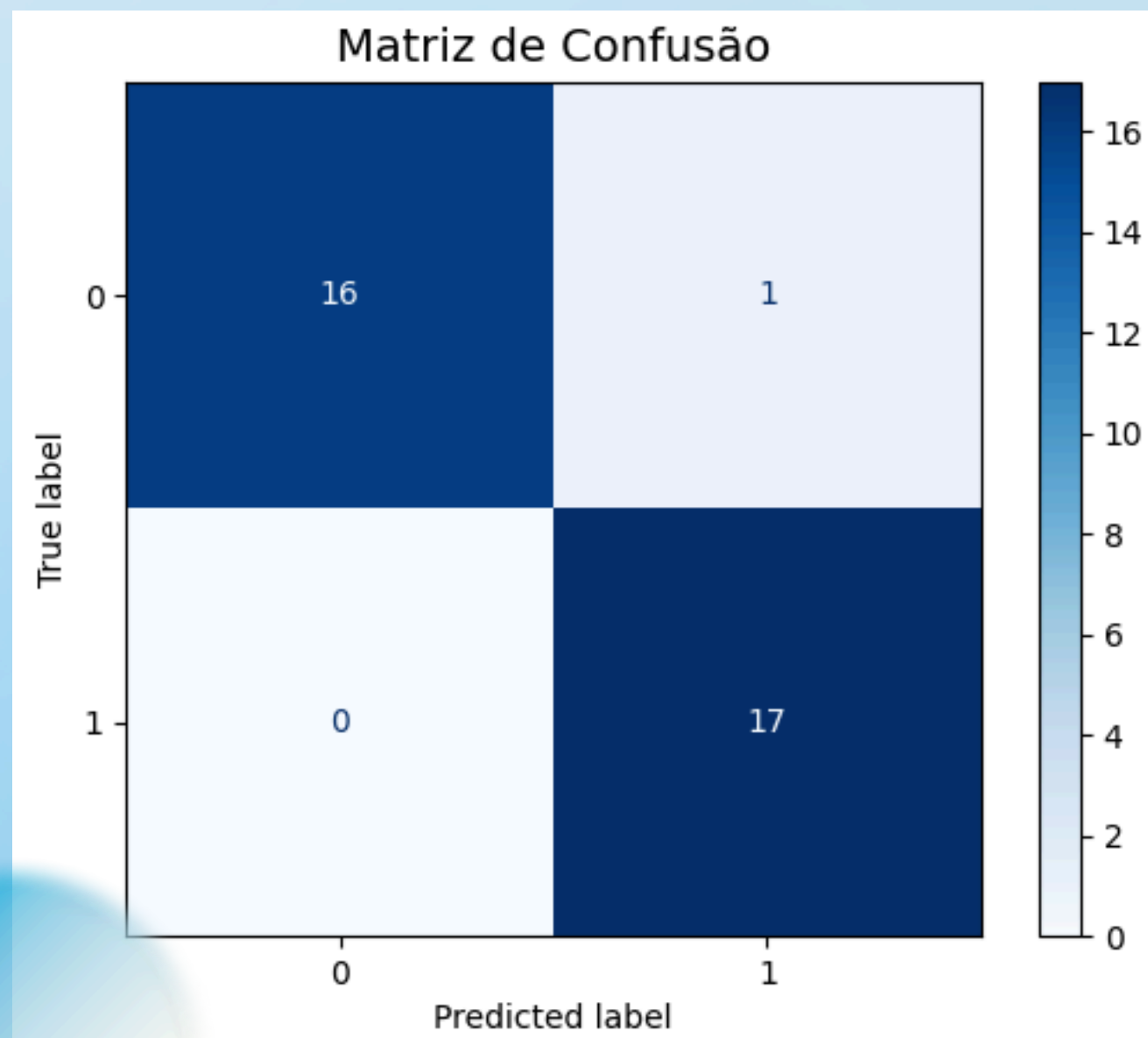


Base de dados Iris

- Teste feito com as classes Versicolor e Virginica por não serem linearmente separáveis.
- Utilizada para validar o desempenho da rede em uma base multidimensional (4D nesse caso).
- Como a convergência é impossível nesse caso, é visível uma grande variação da acurácia no início do treino até a “estabilização” em 97%.



Desempenho na base



- A estabilização da curva de acurácia em 97% se deve ao fato de os dados de teste terem sido classificados “da mesma maneira” (mesma quantidade de erros) a cada avaliação, não que este é realmente o melhor limiar a ser demarcado para estes dados.
- Na matriz de confusão é possível visualizar que o modelo classificou uma amostra como Virginica quando na verdade era versicolor, correspondendo a um Falso Positivo.

Referências

sklearn.metrics. Scikit-learn, 2026. Disponível em: <https://scikit-learn.org/stable/api/sklearn.metrics.html>. Acesso em: 22 de maio de 2026.

matplotlib.pyplot. Matplotlib, 2026. Disponível em: <https://matplotlib.org/stable/gallery/pyplots/index.html>. Acesso em: 22 de maio de 2026.

Material de aula do curso de Sistemas Inteligentes, ministrado pelo professor Dr. Pablo Andretta Jaskoviak

